

Overseas Mobility Management Platform

Databases Mod. 2 — Project Report, A.Y. 2025/2026

Ca' Foscari University of Venice

1. Introduction

This report documents the design and implementation of a web application that manages the administrative phases of a Ca' Foscari **Overseas** mobility: the period before departure, the mobility itself, and the recognition of the exams taken abroad after the return.

The application is a Flask REST API over a PostgreSQL database, accessed through SQLAlchemy, with an Angular single-page front end. Three kinds of user share one record — the *mobility application* — and move it through a fixed lifecycle: the student prepares and submits it, the academic coordinator decides on its academic content, and the Overseas office performs the two administrative checks that open and close the process.

The document is organised as required by the project brief. Section 2 describes the functionalities actually implemented. Section 3 presents the conceptual and logical design of the database. Section 4 shows the most interesting queries in SQL. Section 5 explains the main design choices: integrity policies, authorisation, attachments and application statuses. Section 6 collects the technological choices and the information needed to run the project. The appendix records the contribution of each group member.

Everything described here corresponds to the submitted source code. Where a rule of the brief is enforced somewhere other than the schema, the report says so explicitly rather than implying a constraint that does not exist.

2. Main functionalities

2.1 Roles

The system distinguishes three roles, held by exactly one user each:

Role	Scope of what it can see	Main operations
Student	Only their own applications	Create and revise an application, propose the exam mapping, upload Learning Agreements, record arrival and departure, upload the Transcript of Records, enter grades
Academic coordinator	Only the applications assigned to them	Read the uploaded documents, approve or reject each Learning Agreement version, approve or reject each exam recognition
Overseas office	Every application	Maintain the list of partner institutions, complete the pre-departure check, close the application

A user signs in with email and password; the session is carried by a signed cookie. Every endpoint other than login requires a session, and each group of endpoints requires one specific role.

2.2 Before departure

The student creates an application by entering the academic year, the host institution chosen from the predefined partner list, the expected mobility period (first semester, second semester or full year), the responsible academic coordinator, and an optional note. While the application is still a draft the student may revise those details or withdraw it entirely.

The student then uploads the signed Learning Agreement together with the mapping between the exams to be taken abroad and the courses of the Ca' Foscari study plan. Each mapping row carries the code, title and credits of the foreign course and of the corresponding home course. Uploading sends the application to the coordinator and freezes the mapping: while a version is awaiting a decision, it cannot be edited.

The coordinator reads the uploaded file and approves or rejects the version, recording the decision date and, for a rejection, a motivation. A rejection returns the application to the student, who revises the mapping and uploads a new version. Once a version is approved, the Overseas office performs the pre-departure check, which the system allows only when an approved Learning Agreement with at least one course mapping exists.

2.3 During the mobility

The student records the actual arrival date at the host institution, which starts the mobility, and the actual departure date when it is known. The departure date can never precede the arrival date.

If the exam programme changes, the student proposes a modification: a new Learning Agreement version, with the revised mapping and the updated signed document. The coordinator approves or rejects it exactly as before. An approved modification becomes the mapping in force and the mobility resumes; a rejected one leaves the previously agreed mapping standing, because it was never replaced.

2.4 After the return

The student uploads the Transcript of Records, which opens the recognition phase, and then completes each mapped exam with the grade obtained and the date on which it was passed. When both mobility dates are known, an exam date outside that interval is refused.

The coordinator approves or rejects each exam and its grade, again with a date and, on rejection, a motivation. Only exams belonging to the Learning Agreement version currently in force can be decided.

Finally the Overseas office closes the application. Closure is allowed only when the transcript has been uploaded and every submitted exam has been decided; a rejected recognition is a decision and does not hold the application open, a pending one does. A closed application is final and can no longer be modified.

2.5 Documents and history

Learning Agreements are versioned: every version uploaded for an application is kept, together with its upload timestamp, its decision, its decision date and the motivation of a rejection. All three roles can download any version they are entitled to see, and the transcript likewise.

Every change of application status is recorded with the previous status, the new status and the moment of the change, so the whole lifecycle of an application can be reconstructed.

2.6 Partner institutions

The Overseas office maintains the predefined list of partner institutions, each with name, country, city and an optional contact address. Institutions are deactivated rather than deleted: a deactivated partner disappears from the list students choose from, while past applications keep pointing at it.

3. Conceptual and logical design

3.1 Conceptual schema

The diagram uses the notation of Module 1: rectangles for entities, double rectangles for weak entities, diamonds for relationships, double diamonds for identifying relationships, and (min, max) pairs for cardinality and participation.

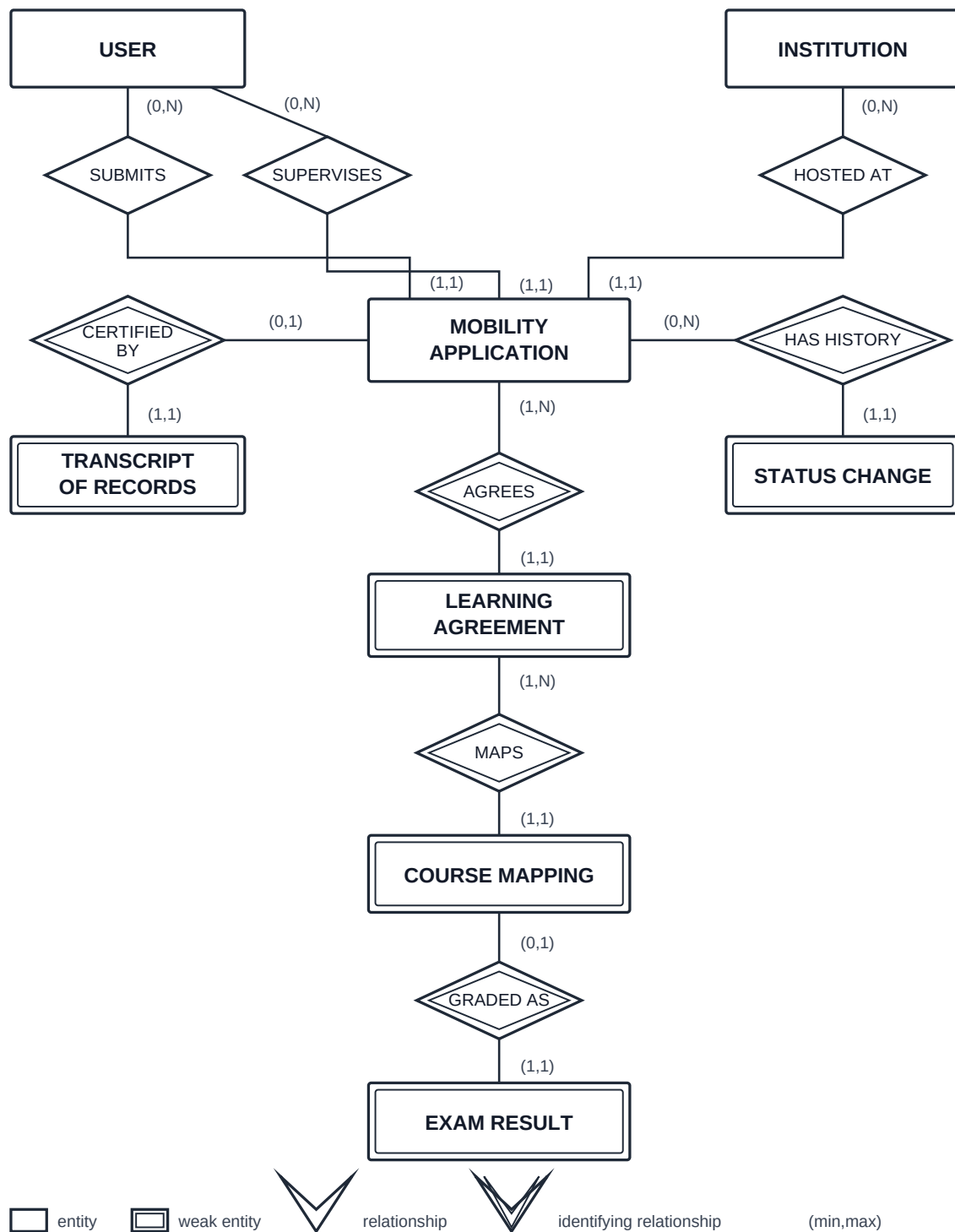


Figure 1 — Conceptual schema (Entity-Relationship diagram).

Entities

USER — a person who signs in: email, password hash, first and last name, role. The three roles share every attribute and differ only in what they may do, so they are one entity with a role attribute rather than three entities. Since an application must reference exactly one student and exactly one coordinator, a generalisation into three entities would have left those two relationships pointing at different entities for no gain.

INSTITUTION — a partner university abroad: name, country, city, optional contact address, and an active flag. The triple (name, country, city) identifies it externally, which allows two campuses of the same university in different cities.

MOBILITY APPLICATION — the centre of the schema: academic year, expected mobility period, optional note, status, and the actual arrival and departure dates, which stay unknown until the student reports them.

STATUS CHANGE — weak entity: the previous status, the new status and the instant of the change.

LEARNING AGREEMENT — weak entity identified by its application and a version number: the stored file, the upload timestamp, the approval status, the decision date and the rejection motivation.

COURSE MAPPING — weak entity belonging to one agreement version: the code, title and credits of the foreign course and of the home course.

TRANSCRIPT OF RECORDS — weak entity: the stored file and its upload timestamp.

EXAM RESULT — weak entity belonging to one course mapping: the grade obtained abroad, the exam date, the recognition status, the decision date and the rejection motivation.

Relationships

Relationship	Between	Cardinality	Participation
SUBMITS	USER (student) — MOBILITY APPLICATION	1 : N	total on the application side
SUPERVISES	USER (coordinator) — MOBILITY APPLICATION	1 : N	total on the application side
HOSTED AT	INSTITUTION — MOBILITY APPLICATION	1 : N	total on the application side
HAS HISTORY	MOBILITY APPLICATION — STATUS CHANGE	1 : N	total on the status-change side
AGREES	MOBILITY APPLICATION — LEARNING AGREEMENT	1 : N	total on the agreement side
MAPS	LEARNING AGREEMENT — COURSE MAPPING	1 : N	total on the mapping side
CERTIFIED BY	MOBILITY APPLICATION — TRANSCRIPT OF RECORDS	1 : 1	total on the transcript side, optional on the application side
GRADED AS	COURSE MAPPING — EXAM RESULT	1 : 1	total on the result side, optional on the mapping side

The optional participations mirror the phases of the process: an application has no transcript until the student returns, and a mapping has no result until the exam is taken and reported.

Conceptual decisions

A modification is a new version, not a new entity. The brief asks that the student may propose changes during the mobility and that a rejection restores the previously agreed mapping. Versioning the Learning Agreement, with each version owning its own mappings, satisfies both without a separate "modification" entity: the mapping in force is the one belonging to the highest approved version, and rejecting a later version leaves the earlier one untouched.

Documents are files. The brief allows the Learning Agreement and the Transcript of Records to be handled as uploaded files, so each carries a path and an upload timestamp rather than a model of its content.

An exam result belongs to a mapping. A grade is always the grade of one agreed exam pair. Attaching it to the mapping makes it automatically belong to the plan version it was agreed under, which is what allows

recognition decisions to be restricted to the version in force.

Status history is stored, not derived. One decision in the workflow — approving a plan modification during an ongoing mobility — depends on whether the pre-departure phase was ever completed, which the current status alone cannot answer.

3.2 Logical schema

The conceptual schema translates into eight tables. Weak entities become tables whose key includes the identifier of the owning entity, or, where a surrogate key was more convenient for the application, a surrogate key plus a uniqueness constraint that preserves the one-to-one or one-to-many meaning.

```
USER_ACCOUNT(user_id, email, password_hash, first_name, last_name, user_role)
INSTITUTION(institution_id, name, country, city, contact_email, is_active)
MOBILITY_APPLICATION(application_id, academic_year, optional_note,
                     expected_mobility_period, status,
                     actual_arrival_date, actual_departure_date,
                     student_id, coordinator_id, host_institution_id)
APPLICATION_STATUS_HISTORY(history_id, application_id, old_status,
                           new_status, changed_at)
LEARNING_AGREEMENT(application_id, version_number, file_path, uploaded_at,
                    approval_status, decision_date, rejection_reason)
COURSE_MAPPING(mapping_id, application_id, version_number,
               foreign_course_code, foreign_course_name, foreign_course_credits,
               home_course_code, home_course_name, home_course_credits)
TRANSCRIPT_OF_RECORDS(transcript_id, application_id, file_path, uploaded_at)
EXAM_RESULT(result_id, mapping_id, foreign_grade, exam_date,
             recognition_status, decision_date, rejection_reason)
```

Four enumerated database types back the status columns, so an unknown value cannot be stored at all and the same vocabulary is shared by the schema, the API and the front end:

Type	Values
user_role	student, coordinator, office_staff
mobility_period	first_semester, second_semester, full_year
application_status	created, waiting_la_approval, pre_departure_completed, mobility_in_progress, under_exam_recognition, closed
approval_status, recognition_status	pending, approved, rejected

Keys and referential integrity

LEARNING_AGREEMENT has the composite primary key (application_id, version_number), which makes version numbers unique within an application by construction. COURSE_MAPPING references that pair as a composite foreign key and additionally keeps a surrogate mapping_id, because EXAM_RESULT refers to a single mapping and the API addresses mappings by identifier.

The one-to-one relationships are enforced by uniqueness: TRANSCRIPT_OF_RECORDS.application_id and EXAM_RESULT.mapping_id are unique.

Deletion behaviour follows the conceptual distinction between what an application *owns* and what it merely *refers to*. Status history, agreements, mappings, transcript and results cascade with the application, because none of them means anything without it. Users and institutions are protected with ON DELETE RESTRICT: an application is an administrative record, so neither the student, nor the coordinator, nor the host institution may be deleted while one exists.

Domain constraints

Constraint	Table	Meaning
ck_mobility_application_academic_year_format	mobility_application	academic_year matches <code>^[0-9]{4}/[0-9]{4}\$</code>
ck_mobility_application_actual_dates	mobility_application	a departure date requires an arrival date and cannot precede it
ck_learning_agreement_version_positive	learning_agreement	version_number > 0
ck_learning_agreement_decision_consistency	learning_agreement	pending has neither decision date nor reason; approved has a date and no reason; rejected has a date and a non-empty reason
ck_exam_result_decision_consistency	exam_result	the same three legal decision shapes
ck_course_mapping_home_credits_positive, ck_course_mapping_foreign_credits_positive	course_mapping	both credit values are positive
uq_course_mapping_plan_course_pair	course_mapping	a course pair cannot repeat inside one plan version
uq_host_institution_location	institution	a partner cannot be entered twice

The two decision-consistency constraints are the most useful of these: three nullable columns (status, date, reason) admit combinations that mean nothing, and encoding the three legal shapes as one CHECK keeps them from drifting apart regardless of which code path writes them.

Choices of type

Credits are `NUMERIC(4,1)` rather than integers, because ECTS values such as 7.5 are common. Grades are free text of up to 50 characters: host institutions grade on incompatible scales (30/30, A–F, pass/fail), and the original mark reported by the transcript is what has to be preserved. Timestamps are `TIMESTAMPZ`; dates that are administrative facts rather than instants — decision dates, exam dates, arrival and departure — are `DATE`.

4. Main queries

The application uses the SQLAlchemy Expression Language through the ORM, so no SQL string is written by hand; the statements below are the SQL the ORM produces for the most interesting cases, with parameters shown as `:name`.

Ownership scoping. Every student-facing lookup is filtered by the signed-in user, so authorisation is part of the query rather than a check performed after loading the row:

```
SELECT *
FROM mobility_application
WHERE application_id = :application_id
AND student_id = :student_id;
```

The coordinator's version filters on `coordinator_id` instead. The office runs the unfiltered query, optionally narrowed by status, which is what the office dashboard uses:

```
SELECT *
FROM mobility_application
WHERE (:status IS NULL OR status = :status)
ORDER BY application_id DESC;
```

The Learning Agreement version in force. Several rules depend on the highest *approved* version rather than the most recent one — an approved modification replaces the plan, a rejected one does not:

```
SELECT la.*
FROM   learning_agreement la
WHERE  la.application_id = :application_id
      AND la.approval_status = 'approved'
ORDER BY la.version_number DESC
LIMIT 1;
```

The mapping currently in force, with its results. This is what the detail page of all three roles shows:

```
SELECT cm.mapping_id,
       cm.foreign_course_code, cm.foreign_course_name, cm.foreign_course_credits,
       cm.home_course_code,   cm.home_course_name,   cm.home_course_credits,
       er.foreign_grade, er.exam_date, er.recognition_status,
       er.decision_date, er.rejection_reason
FROM   course_mapping cm
LEFT JOIN exam_result er ON er.mapping_id = cm.mapping_id
WHERE  cm.application_id = :application_id
      AND cm.version_number = (
          SELECT MAX(version_number)
          FROM   learning_agreement
          WHERE  application_id = :application_id
              AND approval_status = 'approved')
ORDER BY cm.mapping_id;
```

Applications the office may sign off. The pre-departure check is allowed only for an application awaiting a decision outcome that has an approved agreement carrying at least one mapping:

```
SELECT a.application_id, a.academic_year, i.name AS institution
FROM   mobility_application a
JOIN   institution i ON i.institution_id = a.host_institution_id
WHERE  a.status = 'waiting_la_approval'
      AND EXISTS (
          SELECT 1
          FROM   learning_agreement la
          JOIN   course_mapping cm
                ON cm.application_id = la.application_id
                AND cm.version_number = la.version_number
          WHERE  la.application_id = a.application_id
              AND la.approval_status = 'approved');
```

What blocks a closure. An application may be closed only once the transcript is in and no submitted exam is still pending:

```
SELECT a.application_id,
       (t.transcript_id IS NOT NULL) AS has_transcript,
       COUNT(er.result_id) AS submitted,
       COUNT(*) FILTER (WHERE er.recognition_status = 'pending') AS still_pending
FROM   mobility_application a
LEFT JOIN transcript_of_records t ON t.application_id = a.application_id
LEFT JOIN course_mapping cm ON cm.application_id = a.application_id
LEFT JOIN exam_result er ON er.mapping_id = cm.mapping_id
WHERE  a.status = 'under_exam_recognition'
GROUP BY a.application_id, t.transcript_id;
```

The lifecycle of one application. The history table answers both "how did this application get here" and "has it ever been in this phase", the second of which decides whether an approved modification resumes a mobility or waits for the pre-departure check:

```
SELECT old_status, new_status, changed_at
FROM   application_status_history
WHERE  application_id = :application_id
ORDER BY changed_at;
```

5. Main design choices

5.1 Where each rule is enforced

The requirements document lists twenty-six business rules. They are enforced in two places, and the split is deliberate: a rule that constrains one row is a schema constraint, because then no code path can violate it; a rule that spans rows, or depends on who is asking, is enforced in the service layer, because a CHECK constraint cannot express it.

In the schema: exactly one role per user; exactly one student, coordinator and institution per application; the academic-year format; positive credits; unique course pairs within a plan version; unique version numbers; date ordering; the legal combinations of decision status, decision date and motivation; at most one transcript per application and one result per mapping.

In the service layer: that a student reaches only their own applications and a coordinator only those assigned to them; that the second academic year is the first plus one; that the mapping is frozen while a version awaits a decision; that the pre-departure check requires an approved agreement; that an exam date falls inside the mobility period; that recognition requires the transcript and closure requires every result decided; and that status changes follow the permitted workflow.

The service layer validates before writing even where the database would also refuse, so the user receives a specific message — "a rejection must include a reason" — instead of an integrity error.

5.2 Transactions

Every HTTP request is one transaction. Services end their work with `db.session.flush()`, which makes their writes visible to later queries within the same request without deciding that the request as a whole succeeded; the commit or rollback happens once, in an `after_request` hook, and any response of 400 or worse rolls back. An exception that escapes a view rolls back in `teardown_request`.

The effect is that a refused operation leaves nothing behind. Uploading a Learning Agreement, for example, inserts the version, inserts its mappings, changes the application status and appends a history row; if any of those is refused, none of them happened. The one operation with an effect outside the transaction is writing the uploaded file to disk, which is why the mappings are validated *before* the file is written: a rejected submission leaves no orphan upload.

5.3 Roles and authorisation

Authentication is a signed session cookie, marked `HttpOnly` so page scripts cannot read it, with `SameSite` and `Secure` configurable for deployment behind HTTPS. Passwords are hashed with Argon2id; only the encoded hash is stored, and it already contains the salt and the algorithm parameters, so no salt column is needed. Verification is performed by the hashing library, never by an SQL comparison.

Authorisation has two layers. Each endpoint requires one role, checked by a decorator before the view runs. Ownership is then enforced inside the queries: the student's and the coordinator's lookups carry their own identifier as a filter, so an application belonging to somebody else is not found rather than forbidden — which also keeps its existence private from a user with no claim on it. The Overseas office alone runs unscoped queries.

Document downloads follow the same path as the data: each role has its own download endpoint, which first fetches the application through that role's scoped query and only then locates the file. A file path never

comes from the request, so no path traversal is possible, and no student can download another student's Learning Agreement.

5.4 Attachments

Only the path of an uploaded document is stored in the database. Keeping the binaries out of the database keeps dumps and queries small, and the brief explicitly allows the documents to be handled as files.

Files are written under a collision-resistant generated name, so two students uploading LA.pdf cannot overwrite each other, and the original name is never used as a path. On download the file is renamed to something meaningful — `learning-agreement-v2.pdf` — because the stored name carries no information for the person downloading it.

5.5 Application statuses

The status column is an enumerated type, and one module owns every write to it. Services never assign the column directly: they call a `transition` function that refuses any move outside the permitted workflow and appends the corresponding history row in the same operation, so the history cannot drift from the status.

The workflow is the one in the brief, with two additions that the brief implies rather than states. A rejected Learning Agreement returns the application to `created`, which is what lets the student revise the mapping and submit a new version. A modification approved during an ongoing mobility returns the application to `mobility_in_progress` rather than to the pre-departure check, which is decided by asking the history whether the pre-departure phase was ever completed.

5.6 Structure of the code

The backend is layered: routes translate HTTP into calls and back, services own the domain rules, repositories own the queries and the ownership scoping, and models own the schema. Services raise domain errors carrying a stable code and know nothing about HTTP; one module maps each code to a status and a message, so a new rule can be added without touching any route.

The three roles have separate blueprints and separate services, because they have genuinely different rules — but everything they share (locating a document, validating a decision, moving the status) lives in one place rather than being repeated three times.

5.7 Automated tests

The test suite covers the three role services, the HTTP routes together with their authorisation, the database constraints, the seed script, document downloads, and the application wiring. Each test runs inside a transaction that is rolled back afterwards, so tests neither see nor leave behind each other's rows, and the suite points by default at a database of its own so that a bare `pytest` cannot touch development data.

5.8 What the schema deliberately does not contain

For honesty about the delivered state: the database defines no triggers, no additional indexes beyond those implied by primary keys and unique constraints, and no views. The workflow rules that a trigger could enforce are enforced in the single module that owns status changes, and the data volumes of the project do not make index tuning measurable.

6. Additional information

6.1 Technological choices

Layer	Choice	Reason
Database	PostgreSQL 16	Enumerated types, CHECK constraints and TIMESTAMPTZ, as recommended by the brief
Access	SQLAlchemy 2 with Flask-SQLAlchemy 3	ORM and Expression Language, so no SQL dialect is hard-coded
Migrations	Alembic through Flask-Migrate	The schema is versioned with the code
API	Flask 3	Required by the brief
Passwords	argon2-cffi (Argon2id)	Memory-hard hashing, salt embedded in the encoded hash
Cross-origin	Flask-CORS	The front end runs on its own origin during development
Front end	Angular 22, plain CSS	Reuse of the group's experience; no CSS framework
Serving	Gunicorn in the container	The Flask development server is not used for the packaged run
Tests	pytest	Service-level and route-level tests against a real PostgreSQL

6.2 Layout of the submission

services/database/	Flask API: models, routes, services, repositories, migrations, seed data and tests
frontend/	Angular single-page application
docs/	This report and the presentation
docker-compose.yml	PostgreSQL + API + front end

The database documentation — requirements and business rules, conceptual schema, logical schema — is in `services/database/database_docs/`.

6.3 Running the project

With Docker, from the repository root:

```
docker compose up --build
```

The front end is then served on `http://localhost:4200` and the API on `http://localhost:5001/api`. The backend container creates its database, applies the migrations and loads the development data on start.

Without Docker, with PostgreSQL running locally:

```
cd services/database
pip install -r requirements.txt
cp .env.example .env
python -m scripts.init_database
flask --app main_module.py db upgrade -d migrations
python -m seeds.dev_seed
python main_module.py
```

6.4 Demonstration accounts

The seed loads eight partner institutions and five accounts, all sharing the password `password: student@test.com` and `student2@test.com` (students), `lecturer@test.com` and `lecturer2@test.com` (coordinators), and `office@test.com` (Overseas office). The seed matches on natural keys, so running it again updates rather than duplicates.

Appendix — Contribution of each group member

The division of work below follows the commit history of the project repository.

Member	Contribution
Daren Akpinar	Initial structure of the database service; the SQLAlchemy models for users, institutions, applications, Learning Agreements, course mappings and documents; the first table-creation migration; and the database documentation — requirements and business rules, and the first drafts of the conceptual and logical schema.
Umar Akhmaev	Alembic setup and the initial mobility-schema revision; the integrity constraints on the schema, including the decision-consistency CHECKs; the exam-result and status-history models; Flask session authentication with the role decorators and Argon2id password handling; the HTTP layer — the student, coordinator, office, reference and authentication blueprints, their role guards and the shared route helpers; and the database-constraint tests.
Mihail Skorenko	Repository and Docker Compose setup; the Angular front end for all three roles; the backend domain layer — services, repositories with their ownership scoping, serializers, the status-workflow module, document storage and the per-request transaction boundary; the seed script; and the service, route, constraint and download test suite.

Documentation of the project as submitted — this report, the conceptual and logical schema documents and the repository README — was written jointly, from the design decisions each member had made in their own part of the work.